

多轮 LLM Agent 的信用分配，已经成为 2025–2026 年 Agentic RL 最核心的问题之一。  
截至 2026 年 8 月，前沿方法大致可以归纳为：

轨迹广播 → 逐轮奖励 → 层级优势 → 轨迹图比较 → 过程奖励模型 → hindsight/反事

目前还没有公认的最终方案，但研究方向已经比较清晰。

## 一、问题到底是什么？

假设 Agent 执行一个 10 轮任务：

```
t1: 理解用户目标
t2: 搜索商品
t3: 选择错误页面
t4: 返回搜索
t5: 筛选价格
t6: 选择商品
t7: 填写地址
t8: 选择配送
t9: 支付
t10: 任务完成
```

最终奖励：

$$R(\tau) = 1.$$

普通 GRPO 会把同一个优势分配给整条轨迹的所有 token：

$$A_{t,k} = A(\tau), \quad \forall t, k.$$

这里：

- $t$  是交互轮次；
- $k$  是该轮中的 token。于是：
- 正确搜索得到正奖励；
- 错误进入页面也得到正奖励；

- 补救动作得到正奖励；
- 冗余思考同样得到正奖励。如果最终失败，则之前所有正确操作也会被惩罚。真正想估计的是：

$$A_t = Q(s_t, a_t) - V(s_t),$$

即：

在状态  $s_t$  下执行动作  $a_t$ ，相对于其他可选动作，究竟让最终成功概率提高了多少？难点在于只有执行完整轨迹后才能看到结果，而且环境具有：

- 长 horizon；
- 部分可观测；
- 随机工具反馈；
- 自然语言动作空间；
- 错误累积和后续恢复；
- 同一个动作在不同上下文中的价值不同。

## 二、第一类：把轨迹奖励拆成逐轮可验证奖励

### 1. Multi-Turn GRPO / Turn-Level Reward Design

#### Reinforcing Multi-Turn Reasoning in LLM Agents via Turn-Level Reward Design

是早期最直接的工作。它把多轮 Agent 看成真正的 MDP，而不是一个超长 bandit response。每一轮可以获得局部奖励：

$$r_t = r_t^{\text{tool}} + r_t^{\text{format}} + r_t^{\text{progress}} + \mathbf{1}[t = T]r^{\text{outcome}}.$$

然后使用 return：

$$G_t = \sum_{j=t}^T \gamma^{j-t} r_j,$$

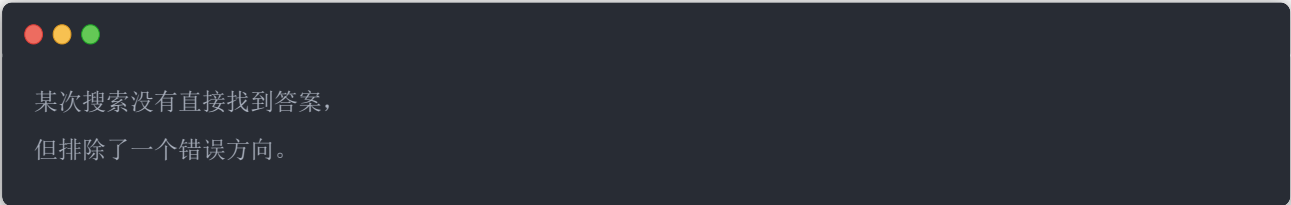
计算逐轮 advantage：

$$A_t = G_t - V(s_t).$$

例如搜索 Agent 可以设计：

| 动作       | 局部信号 |
|----------|------|
| 正确调用搜索工具 | 正    |
| 参数格式合法   | 正    |
| 检索到相关文档  | 正    |
| 重复搜索相同内容 | 负    |
| 工具调用解析失败 | 负    |
| 最终答案正确   | 大正奖励 |

优点是实现简单、信号直接；缺点是必须人工定义局部奖励，而且局部合理不一定导致全局成功。 典型问题是：



从长期看有价值，从局部规则看却可能被判为无效。

---

### 三、第二类：层级 Actor-Critic

---

#### 2. ArCher: token 级和 turn 级分开建模

---

[ArCher: Training Language Model Agents via Hierarchical Multi-Turn RL](#) 是多轮 LLM RL 的重要基础工作。 它将策略分为两个层级：

- 高层：每个 turn 是一个 action；
- 低层：turn 内部由 token 逐个生成。 可写成：

$$a_t = (y_{t,1}, \dots, y_{t,K_t}).$$

高层 critic 估计：

$$Q_{\text{turn}}(s_t, a_t),$$

再将 turn advantage 传递到该轮 token:

$$A_{t,k} \approx A_{\text{turn}}(s_t, a_t).$$

相较整轨迹广播，它至少能够区分不同 turn；但同一 turn 内的所有 token 仍共享近似信用。这一层级结构是目前不少多轮方法的基础：

$$\text{trajectory} \rightarrow \text{turn} \rightarrow \text{token}.$$

主要问题是 critic 很难训练：自然语言状态巨大、轨迹长，而且早期状态的 value target 方差很高。

---

## 四、第三类：使用训练期 privileged information 的 critic

---

### 3. SWEET-RL: Actor 看不到答案，Critic 可以看完整未来

---

SWEET-RL 提出 asymmetric/privileged critic。推理时 Actor 只能看到：

$$O_{\leq t}.$$

但训练时 Critic 可以额外看到：

- ground-truth 目标；
- 完整未来轨迹；
- 环境内部状态；
- 最终结果；
- 用户协作者的隐藏信息。因此：

$$V_{\phi}^{\text{privileged}} = V_{\phi}(s_t, \tau_{t:T}, g),$$

其中  $g$  是最终目标或答案。这个 critic 可以事后判断：

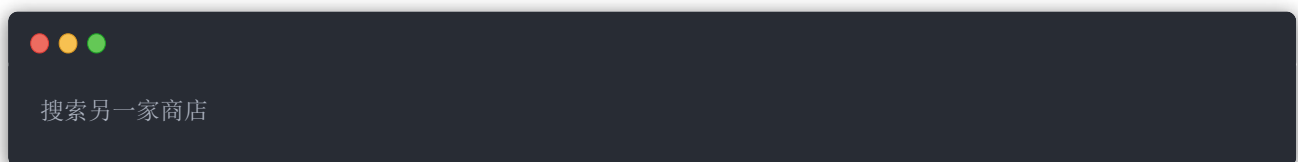
- 当前动作是否推动目标；
- 当前动作是否制造了后续障碍；
- 某次失败是否实际上源自更早的错误；
- 某次修复是否恢复了轨迹。SWEET-RL 在协作式编程和前端设计 ColBench 上，相对多轮 RL 基线获得约 6 个百分点提升。优点：

- 逐轮信号比 outcome reward 稠密；
- 利用训练期信息，不增加推理成本；
- 对协作和开放式任务较适合。 风险：
- critic 可能利用未来信息学会 “事后诸葛亮” 式捷径；
- privileged 信息与实际 actor state 差距太大时会产生偏差；
- critic 错误会系统性污染 policy。

## 五、第四类：Agent Process Reward Model

### 4. AgentPRM: 用 Promise 与 Progress 评价每一步

**AgentPRM: Process Reward Models for LLM Agents via Step-Wise Promise and Progress** 数学 PRM 往往判断某一步 “对或错”，但 Agent action 很难这样标记：



它可能没有立刻完成任务，但会提高未来成功概率。 AgentPRM 因此使用两个概念。

#### Promise

当前状态未来能够成功的概率：

$$P_t \approx P(R = 1 \mid s_t).$$

#### Progress

当前动作使成功概率提高了多少：

$$\Delta P_t = P_{t+1} - P_t.$$

于是某一步的过程奖励可以写成：

$$r_t^{\text{process}} = \alpha P_{t+1} + \beta (P_{t+1} - P_t).$$

例如：

- 打开正确商品页：promise 高、progress 正；
- 原地重复操作：promise 不变、progress 接近 0；
- 删除正确筛选条件：promise 下降、progress 负；
- 看似无关但排除错误方向：可能产生正 progress。论文使用 TD estimation 与 GAE 构造训练标签：

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t),$$

$$A_t^{\text{GAE}} =$$

$$\sum_{l=0}^{T-t} (\gamma \lambda)^l \delta_{t+l}.$$

论文报告 AgentPRM 相对部分 rollout 基线可提高超过 8× 的计算效率，并适合 test-time search。

## 5. OPRL/iStar: 从轨迹偏好反推出隐式逐步奖励

**Online Process Reward Learning for Agentic RL** 不依赖人工 step labels。假设有成功轨迹和失败轨迹：

$$\tau^+ \succ \tau^-.$$

使用 trajectory-level DPO 训练隐式 PRM：

$$L_{\text{pref}} = -\log \sigma \left( \beta \left[ \log \frac{\pi_{\phi}(\tau^+)}{\pi_{\text{ref}}(\tau^+)} - \log \frac{\pi_{\phi}(\tau^-)}{\pi_{\text{ref}}(\tau^-)} \right] \right).$$

由于轨迹 log probability 可以分解：

$$\log \pi_{\phi}(\tau) = \sum_t \log \pi_{\phi}(a_t | s_t),$$

因此可以提取隐式 step reward：

$$r_t^{\text{implicit}} = \beta \log \frac{\pi_{\phi}(a_t | s_t)}{\pi_{\text{ref}}(a_t | s_t)}.$$

再把它与 outcome advantage 结合：

$$A_t = A_t^{\text{outcome}} + \lambda A_t^{\text{process}}.$$

优点是：

- 不需要逐步人工标注；
- 不要求不同 rollout 访问完全相同的状态；
- 过程奖励与 policy 一起在线演化；
- 理论上可解释为 potential-based shaping。问题是隐式奖励代表“偏好模型更喜欢这个动作”，未必等同于真实因果贡献。

## 六、第五类：同状态下的局部反事实比较

### 6. GiGPO：宏观轨迹组 + 微观状态组

[Group-in-Group Policy Optimization](#) 是 critic-free 信用分配的代表方法。[实现](#) 它构造两层组。

#### 宏观组：完整轨迹比较

同一个 prompt 采样多条轨迹：

$$\{\tau_1, \tau_2, \dots, \tau_G\},$$

计算 episode advantage：

$$A_i^{\text{macro}} = \frac{R_i - \mu_R}{\sigma_R}.$$

#### 微观组：相同 anchor state 下的动作比较

如果多条轨迹访问相同或可匹配的状态：

$$s_t^{(i)} \approx s_{t'}^{(j)},$$

则把这些状态后的不同动作组成局部 group：

$$\mathcal{G}(s) = \{a^{(1)}, a^{(2)}, \dots, a^{(m)}\}.$$

根据这些动作最终导向的 return 计算：

$$A^{\text{micro}}(s, a) = \frac{R(s, a) - \mu_{\mathcal{G}(s)}}{\sigma_{\mathcal{G}(s)}}.$$

最终：

$$A_{i,t} = A_i^{\text{macro}} + \lambda A_{i,t}^{\text{micro}}.$$

这近似回答了反事实问题：

在几乎相同的状态下，选择动作 A 还是 B，哪一个更可能成功？它无需 critic、无需额外 rollout，在 ALFWorld 和 WebShop 上相对 GRPO 分别提升超过 12% 和 9%。局限是自然语言 Agent 很少访问字面完全一致的状态，需要可靠的 state matching。

## 七、第六类：轨迹图上的分支归因

### 7. SALT: 将多条 rollout 合并为 trajectory graph

**SALT: Step-level Advantage Assignment via Trajectory Graph** 将同一 prompt 的多条轨迹合并成图。假设：

共同前缀 S0

├─ 动作 A → S1 → 成功

├─ 动作 B → S2 → 失败

└─ 动作 C → S3 → S4 → 成功

在共同前缀之前，很难区分轨迹；在分叉点之后，可以利用不同分支的 outcome 估计动作贡献。设节点  $v$  的后续成功率为：

$$V(v) = \frac{\#\{\text{从 } v \text{ 出发的成功轨迹}\}}{\#\{\text{从 } v \text{ 出发的全部轨迹}\}}.$$

则边或 action 的信用可以近似为：



$$A(v \rightarrow u) = V(u) - V(v).$$

于是：

- 共同出现于成功和失败轨迹的前缀不会获得极端信用；
- 直接导致失败分支的动作得到负信用；
- 将失败路径恢复到成功路径的动作得到正信用。 SALT 不需要额外 reward model、监督或 rollout，可作为 GRPO/RLOO 的插件。 相较 GiGPO：
- GiGPO 强调相同 anchor state 的局部分组；
- SALT 显式利用共享前缀、分叉和汇合构造轨迹图；
- SALT 对状态重复较少但轨迹结构可对齐的任务更自然。

## 八、第七类：Hindsight 信用分配

### 8. HCAPO：看完整结果后重新判断哪些步骤关键

[Hindsight Credit Assignment for Long-Horizon LLM Agents](#) 是 2026 年非常值得关注的工作。 普通 value-free 方法试图仅从当前状态和有限 rollout 估计：

$$Q(s_t, a_t).$$

HCAPO 则让 LLM post-hoc critic 在知道未来结果后进行 hindsight reasoning：

$$q_t^{\text{hindsight}} = Q_\psi(s_t, a_t \mid \tau_{t+1:T}, R).$$

它会回答类似问题：

- 如果没有这个动作，最终结果是否仍可能成功？
- 这个动作是关键决策还是冗余动作？
- 当前失败是它直接造成的，还是由更早动作埋下？
- 这个动作是否修复了此前错误？ 然后构造 multi-scale advantage，将：
- 全局 trajectory advantage；
- 局部 hindsight credit；
- 关键状态校正； 结合起来：

$$A_t^{\text{HCAPO}} = A^{\text{global}} + \lambda_1 A_t^{\text{hindsight}} + \lambda_2 A_t^{\text{critical}}.$$

论文还加入 temporal smoothing，避免只奖励最后一步。例如：

拿起杯子 → 找到水槽 → 清洗杯子

LLM critic 很容易只认为最后的“清洗”有用，而忽略前面的导航和准备。Temporal smoothing 会把部分信用向依赖链上游传播。论文使用 Qwen2.5-7B-Instruct，在 WebShop 和 ALFWorld 上相对 GRPO 分别提升约 7.7% 和 13.8%。这是目前从相关性比较走向“事后语义因果分析”的代表方向。

## 九、第八类：反事实干预

这一方向比 hindsight 更进一步。普通 hindsight 问：

看完结果，这一步是否重要？反事实信用分配则问：如果把这一步替换掉，而保持其他条件尽量不变，最终结果会怎样？理想的 action credit 是：

$$C_t = \mathbb{E}[R \mid do(a_t)] - \mathbb{E}[R \mid do(a'_t)],$$

或者：

$$C_t = R(\tau) - R(\tau_{-t}),$$

其中  $\tau_{-t}$  表示删除或替换第  $t$  步后的反事实轨迹。2026 年的 C3、CCPO 及相关方法开始让模型：

1. 找出疑似关键步骤；
2. 为该步骤生成替代动作；
3. 从替代动作继续 rollout 或让 critic 预测结果；
4. 比较原始与反事实 outcome；
5. 将差值作为该步骤信用。优点是更接近因果贡献；缺点也非常明显：
  - 每个 turn 都重 rollout 成本极高；
  - 环境随机性使反事实比较方差很大；
  - 删除一个动作后，后续原始轨迹可能不再合法；

- LLM critic 模拟的反事实可能不忠实。因此当前实用方法通常是：  
先筛选少量关键 turn → 只对这些 turn 做反事实评估。

## 十、第九类：Checklist / Rubric 过程奖励

### 9. CM2：用细粒度检查表代替单一开放式评分

#### CM2: RL with Checklist Rewards for Multi-Turn and Multi-Step Agentic Tool Use

对于开放式 Agent 任务，很难定义唯一正确轨迹。CM2 将每一轮预期行为拆成多个二元条件：

是否识别用户目标？

是否请求缺失参数？

是否选择正确工具？

是否正确引用工具返回？

是否避免重复调用？

是否在完成后停止？

每个 criterion 带有证据定位和结构化元数据。奖励可以写成：

$$r_t = \sum_{k=1}^K w_k c_{t,k}, \quad c_{t,k} \in \{0,1\}.$$

论文称其为：

sparse reward assignment, dense evaluation criteria。即不必每个 token 都产生奖励，但评估最终或每轮行为时使用非常细的 checklist。优点是可解释、比单个 LLM judge 分数稳定；缺点是 checklist 设计和生成本身可能引入偏差。

## 十一、从方法角度怎么比较？

| 方法             | 信用粒度           | 额外模型              | 额外 rollout | 主要优点                | 主要问题                  |
|----------------|----------------|-------------------|------------|---------------------|-----------------------|
| GRPO outcome   | 轨迹             | 否                 | 否          | 简单稳定                | 所有步骤同奖同罚              |
| MT-GRPO        | Turn           | 可选                | 否          | 容易实现                | 依赖人工局部奖励              |
| ArCHer         | Turn + token   | Critic            | 否          | 层级建模自然              | critic 难训练            |
| SWEET-RL       | Turn           | Privileged critic | 否          | 利用完整未来信息            | 信息泄漏和 critic 偏差       |
| AgentPRM       | Step           | PRM               | 训练 PRM 需要  | Promise/Progress 合理 | PRM 可能 reward hacking |
| OPRL/iStar     | Step           | 隐式 PRM            | 否          | 不要 step labels      | 偏好不等于因果贡献             |
| GiGPO          | Episode + Step | 否                 | 否          | critic-free、开销低     | 依赖重复/可匹配状态            |
| SALT           | 轨迹图边           | 否                 | 否          | 利用分叉结构              | 图对齐质量重要               |
| HCAPO          | Step/Turn      | LLM critic        | 可少量增加      | 能识别前因后果             | critic 成本和幻觉          |
| Counterfactual | Step/Turn      | 可选                | 通常需要       | 最接近因果信用             | 成本最高                  |
| CM2            | Criterion/Turn | Judge             | 否          | 可解释、开放任务可用          | checklist 构造偏差        |

## 十二、目前最前沿的几个趋势

### 1. 从“奖励分解”转向“反事实贡献”

早期方法问：

这一轮看起来好不好？

最新方法开始问：

没有这一轮，最终结果会发生什么变化？

这是从过程评分走向因果归因。

---

## 2. critic-free 与 critic-based 会长期并存

---

critic-free 方法：

- GiGPO;
- SALT;
- trajectory grouping。优点是稳定、低成本、不引入 evaluator bias。critic-based 方法：
- SWEET-RL;
- AgentPRM;
- HCAPO;
- counterfactual critic。它们对开放、随机、长 horizon 任务更灵活，但必须解决 critic 可信度。

---

## 3. 多尺度 advantage 比单一粒度更合理

---

当前比较有前景的形式是：

$$A_{t,k} = \alpha A^{\text{episode}} + \beta A_t^{\text{turn}} + \gamma A_{t,k}^{\text{token}}.$$

三层分别负责：

- Episode：是否完成最终目标；
- Turn：当前工具调用或决策是否有贡献；
- Token：该轮中参数、实体和关键决策 token 是否正确。只用 token reward 容易方差过高；只用 episode reward 又太粗，因此多尺度融合更现实。

---

## 4. 过程奖励不再等于“这一步正确吗”

---

Agent action 更适合两个标准：

$$\text{Promise}(s_t) = P(\text{eventual success} \mid s_t),$$

$$\text{Progress}_t = \text{Promise}(s_{t+1}) - \text{Promise}(s_t).$$

这比逐步判定“正确/错误”更符合搜索、编程、网页操作等任务。

---

## 5. 信用分配正在与蒸馏结合

---

多轮 OPD/自蒸馏可以让教师在学生实际访问的状态上给出 token 级方向：

$$D_{\text{KL}}(\pi_T(\cdot \mid s_t) \parallel \pi_S(\cdot \mid s_t)).$$

教师—学生 logit gap 可作为局部信用信号。但它存在：

- 教师风格漂移；
  - 教师未必理解偏离分布的学生状态；
  - 正确的学生新路径可能被教师覆盖。因此最新方法会把蒸馏、outcome 和反事实结合，而不是无条件模仿教师。
- 

## 十三、我建议优先阅读的论文

---

如果只读 6 篇：

1. [Reinforcing Multi-Turn Reasoning via Turn-Level Reward Design](#)：最基础的 turn-level 方法；
  2. [GiGPO](#)：critic-free 局部比较；
  3. [SWEET-RL](#)：privileged critic；
  4. [AgentPRM](#)：Promise/Progress；
  5. [SALT](#)：trajectory graph；
  6. [HCAPO](#)：hindsight 和多尺度 advantage。系统性综述可以看：[From Reasoning to Agentic: Credit Assignment in Reinforcement Learning for Large Language Models](#)
- 

## 十四、如果应用到多轮 CUDA Kernel 优化

---

CUDA 优化轨迹可能是：

```
生成 Kernel
→ 编译失败
→ 修复类型错误
→ 正确性失败
→ 修复边界条件
→ profile
→ 改共享内存
→ bank conflict 变严重
→ 调整 layout
→ 获得最终加速
```

最合理的信用分配不是仅用最终 speedup，而是：

$$A_t = \alpha A_t^{\text{feasibility}} + \beta A_t^{\text{correctness}} + \gamma A_t^{\text{performance}} + \delta A_t^{\text{hindsight}}.$$

其中：

- feasibility：是否编译、是否能运行；
- correctness：通过多少测试；
- performance：延迟相对上一步的变化；
- hindsight：最终看，这次修改是不是后续加速的必要前提。例如“先把 Kernel 修到正确”本身可能暂时变慢，但它是后续性能优化的必要条件。若只看单步 latency difference，会错误惩罚这一步；若只看最终结果，又会错误奖励中途造成 bank conflict 的修改。适合的组合是：

|                                        |
|----------------------------------------|
| 阶段可验证奖励 + SALT/GiGPO 分支比较 + HCAPO 事后归因 |
|----------------------------------------|

也就是：能客观验证的编译、正确性、延迟直接评分；多个优化轨迹的共同前缀和分叉用轨迹图比较；无法直接判断的准备性修改再由 hindsight critic 归因。